

LABSHEET 7

DISHA SAMANTA

CU24250227

ROLL NO 20

SECTION : B

Import the required Python libraries and load the dataset into a Pandas DataFrame.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

Select a suitable dataset such as the Mall Customer Segmentation Dataset, Online Retail Dataset, or Wholesale Customers Dataset.

```
from google.colab import files
```

```
uploaded = files.upload()
```

Choose files Mall_Customers.csv

Mall_Customers.csv(text/csv) - 4286 bytes, last modified: 07/09/2026 - 100% done

Saving Mall_Customers.csv to Mall_Customers.csv

```
import io
if 'Mall_Customers.csv' in uploaded:
    filename = 'Mall_Customers.csv'
    df = pd.read_csv(io.BytesIO(uploaded[filename]))
    display(df.head())
else:
    print("Error: 'Mall_Customers.csv' not found in uploaded files. Please upload the correct file using the previous cell.")
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)	
0	1	Male	19	15	39	
1	2	Male	21	15	81	
2	3	Female	20	16	6	
3	4	Female	23	16	77	
4	5	Female	31	17	40	

Perform data preprocessing by handling missing values, removing duplicates, and selecting relevant features such as Age, Annual Income, and Spending Score.

```
print("Shape of dataset:", df.shape)

print("\nColumns:")
print(df.columns)

print("\nMissing values:")
print(df.isnull().sum())
```

Shape of dataset: (200, 5)

Columns:
Index(['CustomerID', 'Genre', 'Age', 'Annual Income (k\$)',
 'Spending Score (1-100)'],
 dtype='object')

Missing values:
CustomerID 0
Genre 0
Age 0
Annual Income (k\$) 0
Spending Score (1-100) 0
dtype: int64

```
print("Duplicates before:", df.duplicated().sum())

df = df.drop_duplicates()
```

```
print("Duplicates after:", df.duplicated().sum())
```

```
Duplicates before: 0
Duplicates after: 0
```

```
print("Actual column names in your dataset:")
print(df.columns.tolist())
```

```
Actual column names in your dataset:
['CustomerID', 'Genre', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)']
```

Double-click (or enter) to edit

```
X = df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]

X.head()
```

	Age	Annual Income (k\$)	Spending Score (1-100)	
0	19	15	39	
1	21	15	81	
2	20	16	6	
3	23	16	77	
4	31	17	40	

Next steps: [Generate code with X](#) [New interactive sheet](#)

Normalize or standardize the selected features to ensure that all variables contribute equally during clustering.

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(
    X_scaled,
    columns=X.columns
)

X_scaled.head()
```

	Age	Annual Income (k\$)	Spending Score (1-100)	
0	-1.424569	-1.738999	-0.434801	
1	-1.281035	-1.738999	1.195704	
2	-1.352802	-1.700830	-1.715913	
3	-1.137502	-1.700830	1.040418	
4	-0.563369	-1.662660	-0.395980	

Next steps: [Generate code with X_scaled](#) [New interactive sheet](#)

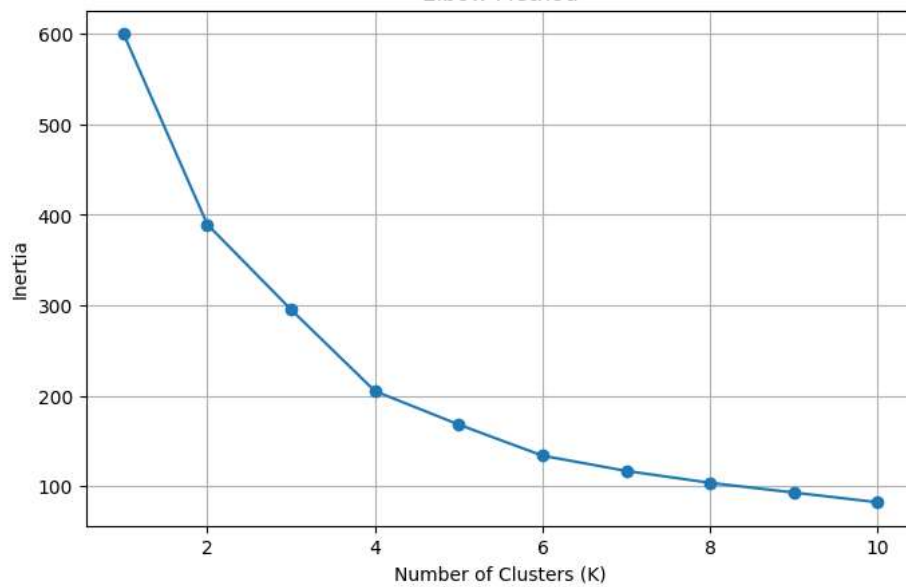
STEP 10 — Silhouette Score

```
inertia = []

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), inertia, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.grid()
plt.show()
```

Elbow Method



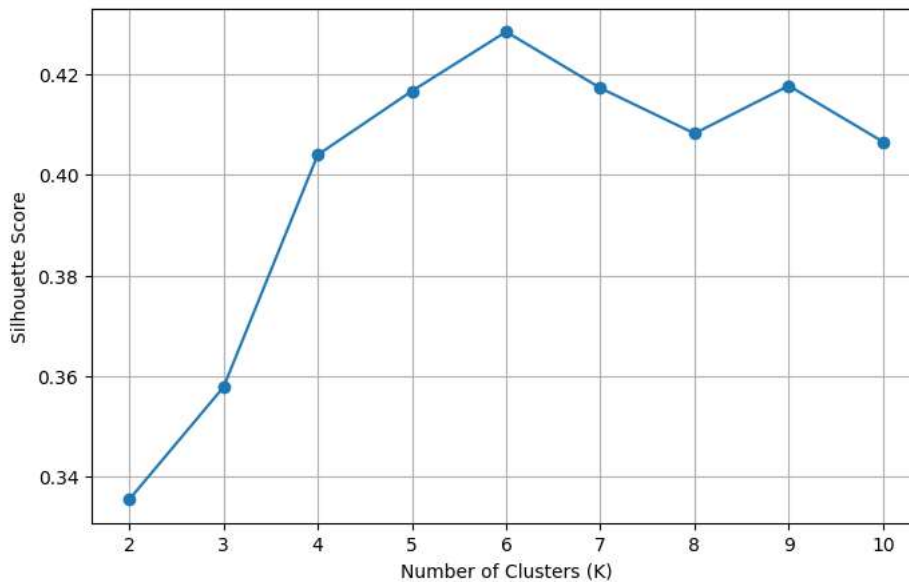
```
silhouette_scores = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)

    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)

plt.figure(figsize=(8, 5))
plt.plot(range(2, 11), silhouette_scores, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score for Different K Values")
plt.grid()
plt.show()
```

Silhouette Score for Different K Values



STEP 11 — Print Silhouette Scores

```
for k, score in zip(range(2, 11), silhouette_scores):
    print(f"K = {k}, Silhouette Score = {score:.3f}")
```

```
K = 2, Silhouette Score = 0.335
K = 3, Silhouette Score = 0.358
K = 4, Silhouette Score = 0.404
K = 5, Silhouette Score = 0.417
K = 6, Silhouette Score = 0.428
K = 7, Silhouette Score = 0.417
K = 8, Silhouette Score = 0.408
K = 9, Silhouette Score = 0.418
K = 10, Silhouette Score = 0.407
```

```
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)

df['Cluster'] = kmeans.fit_predict(X_scaled)

df.head()
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)	Cluster	
0	1	Male	19	15	39	1	
1	2	Male	21	15	81	1	
2	3	Female	20	16	6	0	
3	4	Female	23	16	77	1	
4	5	Female	31	17	40	1	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

STEP 13 — Get Cluster Centroids

```
centers = scaler.inverse_transform(kmeans.cluster_centers_)

centers_df = pd.DataFrame(
    centers,
    columns=['Age', 'Annual Income (k$)', 'Spending Score (1-100)']
)

centers_df
```

	Age	Annual Income (k\$)	Spending Score (1-100)	
0	46.250000	26.750000	18.350000	
1	25.185185	41.092593	62.240741	
2	32.875000	86.100000	81.525000	
3	39.871795	86.102564	19.358974	
4	55.638298	54.382979	48.851064	

Next steps:

[Generate code with centers_df](#)

[New interactive sheet](#)

STEP 14 — Visualize Clusters

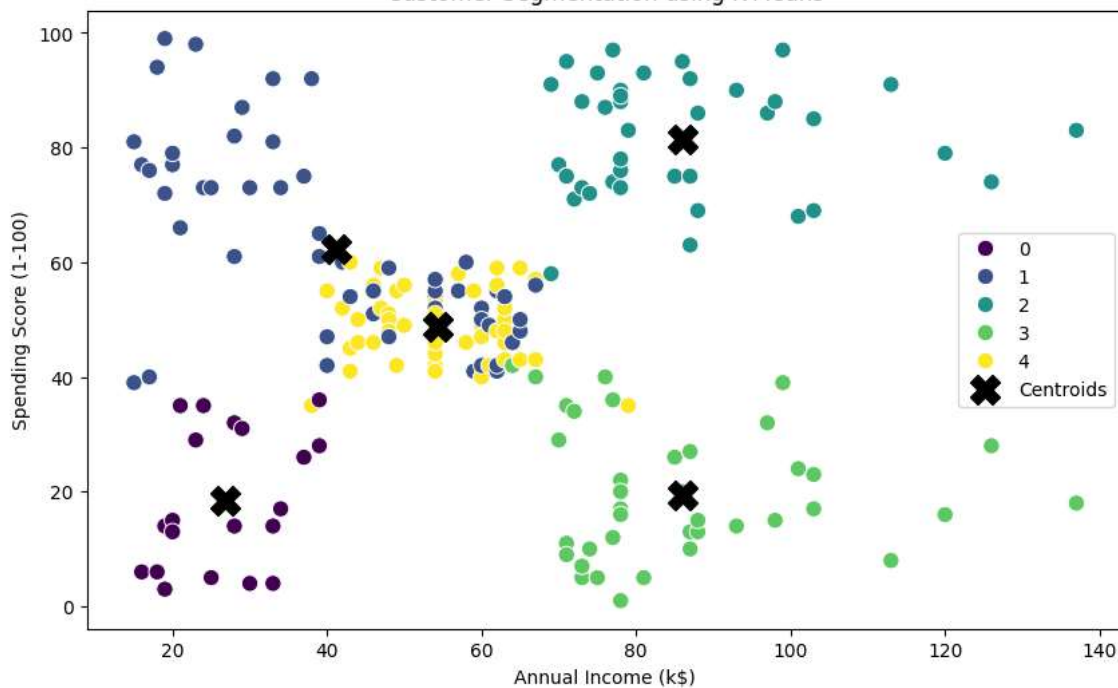
```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster',
    palette='viridis',
    s=80
)

plt.scatter(
    centers_df['Annual Income (k$)'],
    centers_df['Spending Score (1-100)'],
    marker='X',
    s=250,
    color='black',
    label='Centroids'
)

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.legend()
plt.show()
```

Customer Segmentation using K-Means



STEP 15 — Analyze Each Customer Segment

```
cluster_summary = df.groupby('Cluster')[
    ['Age', 'Annual Income (k$)', 'Spending Score (1-100)']
].mean().round(2)
```

cluster_summary

	Age	Annual Income (k\$)	Spending Score (1-100)
Cluster			
0	46.25	26.75	18.35
1	25.19	41.09	62.24
2	32.88	86.10	81.53
3	39.87	86.10	19.36
4	55.64	54.38	48.85

Next steps:

[Generate code with cluster_summary](#)

[New interactive sheet](#)

STEP 16 — Give Meaningful Names to Clusters

```
for cluster in cluster_summary.index:
    income = cluster_summary.loc[cluster, 'Annual Income (k$)']
    spending = cluster_summary.loc[cluster, 'Spending Score (1-100)']

    if income >= 60 and spending >= 60:
        segment = "High Income - High Spending"
    elif income >= 60 and spending < 60:
        segment = "High Income - Low Spending"
    elif income < 60 and spending >= 60:
        segment = "Low Income - High Spending"
    else:
        segment = "Low Income - Low Spending"

    print(f"Cluster {cluster}: {segment}")
```

Cluster 0: Low Income - Low Spending
 Cluster 1: Low Income - High Spending
 Cluster 2: High Income - High Spending
 Cluster 3: High Income - Low Spending
 Cluster 4: Low Income - Low Spending

STEP 17 — Customer Count in Each Cluster

```
print(df['Cluster'].value_counts().sort_index())
```

Cluster
 0 20
 1 54
 2 40

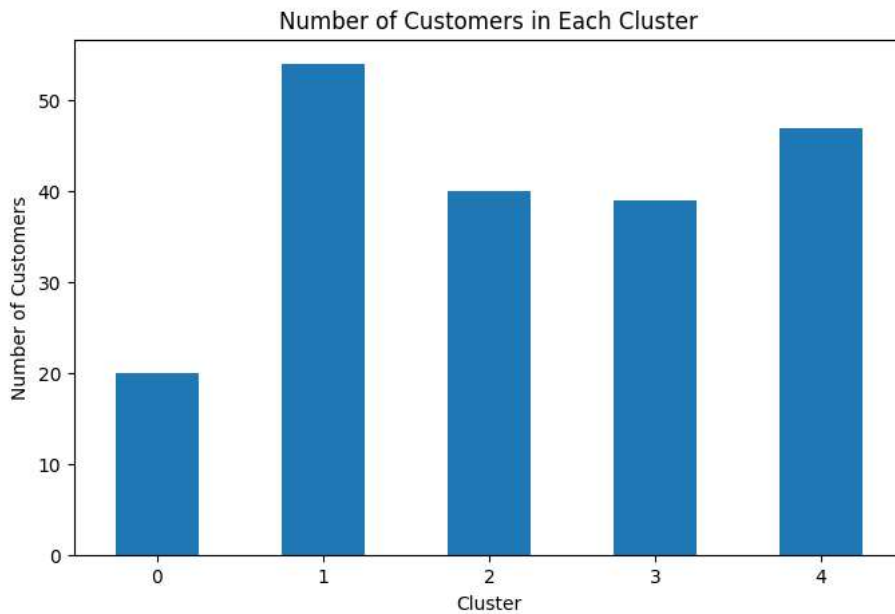
```
3      39
4      47
Name: count, dtype: int64
```

STEP 18 — Bar Graph of Customer Segments

```
plt.figure(figsize=(8, 5))

df['Cluster'].value_counts().sort_index().plot(kind='bar')

plt.xlabel("Cluster")
plt.ylabel("Number of Customers")
plt.title("Number of Customers in Each Cluster")
plt.xticks(rotation=0)
plt.show()
```



STEP 19 — 3D Visualization

```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(10, 7))

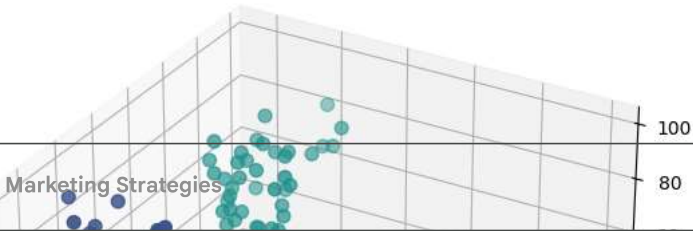
ax = fig.add_subplot(111, projection='3d')

ax.scatter(
    df['Age'],
    df['Annual Income (k$)'],
    df['Spending Score (1-100)'],
    c=df['Cluster'],
    s=50
)

ax.set_xlabel('Age')
ax.set_ylabel('Annual Income (k$)')
ax.set_zlabel('Spending Score')
ax.set_title('3D Customer Segmentation')

plt.show()
```

3D Customer Segmentation



STEP 20 — Marketing Strategies

```
strategies = {
    "High Income - High Spending":
        "Premium products, loyalty programs and exclusive offers.",

    "High Income - Low Spending":
        "Personalized recommendations, discounts and product demonstrations.",

    "Low Income - High Spending":
        "Affordable products, discounts and attractive promotional offers.",

    "Low Income - Low Spending":
        "Budget products, basic offers and awareness campaigns."
}

for strategy, action in strategies.items():
    print(strategy, "→", action)
```

High Income - High Spending → Premium products, loyalty programs and exclusive offers.
High Income - Low Spending → Personalized recommendations, discounts and product demonstrations.
Low Income - High Spending → Affordable products, discounts and attractive promotional offers.
Low Income - Low Spending → Budget products, basic offers and awareness campaigns.

STEP 21 — Final Result

```
print("===== EXPERIMENT 7 RESULT =====")

print("Algorithm: K-Means Clustering")
print("Number of Clusters: 5")

print("\nCluster Distribution:")
print(df['Cluster'].value_counts().sort_index())

print("\nCluster Characteristics:")
print(cluster_summary)

print("\nExperiment completed successfully.")
```

```
===== EXPERIMENT 7 RESULT =====
Algorithm: K-Means Clustering
Number of Clusters: 5

Cluster Distribution:
Cluster
0      20
1      54
2      40
3      39
4      47
Name: count, dtype: int64

Cluster Characteristics:
      Age  Annual Income (k$)  Spending Score (1-100)
Cluster
0      46.25             26.75             18.35
1      25.19             41.09             62.24
2      32.88             86.10             81.53
3      39.87             86.10             19.36
4      55.64             54.38             48.85
```